Node Calibration Windows Application

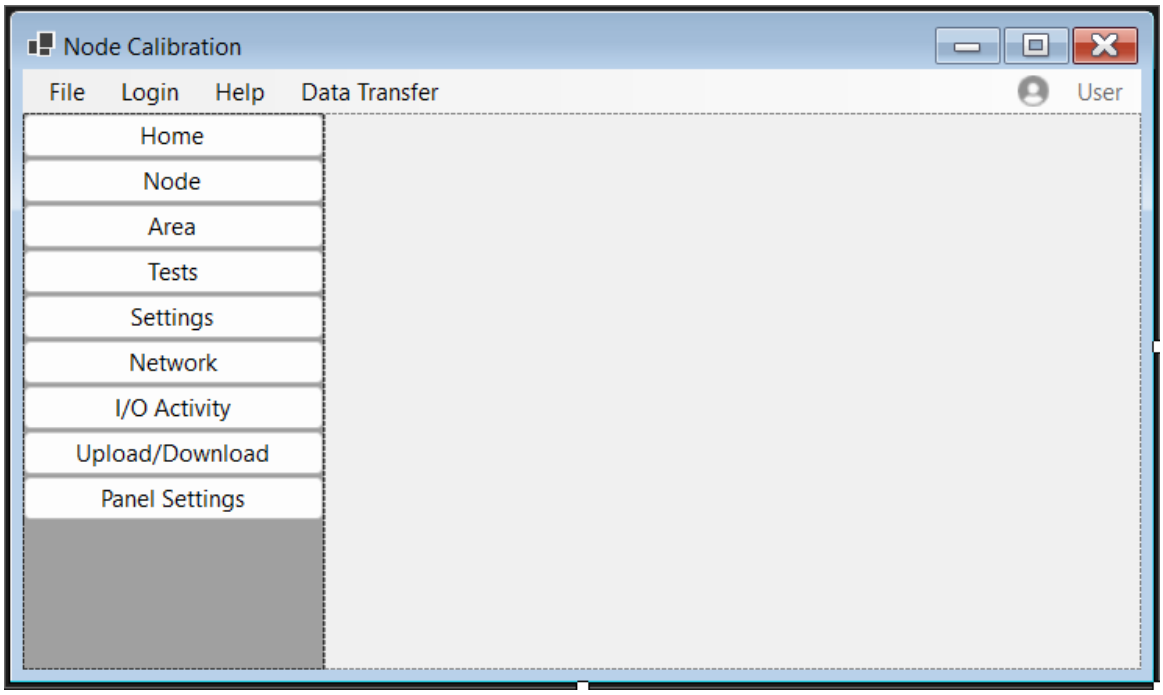
1. Introduction

This document outlines the architectural design and functional requirements for the Node Calibration Windows Desktop Application, developed using C# and the .NET Framework in Microsoft Visual Studio. The application serves as a central interface for [describe primary purpose: e.g., hardware testing, network diagnostics, data monitoring].

2. User Interface (UI) Layout Overview

Region	**Location**	**Purpose**
Navigation Bar	Top-Right	Provides access to global application commands (File, Save, Help, Login).
Navigation Panel	Left Side (Vertical)	Contains command buttons for switching between major functional modules.
Main Content Area	Centre (Remaining Space)	Dynamically displays the active module view or the company logo on startup.

2.1. Form Snapshot



3. Detailed Component Specifications

3.1. Top Navigation Bar (Menu Bar)

This component is positioned at the top of the application window. It provides standard Windows application controls.

Menu Item	**Sub-Items**	**Functional Description**
File	New	Opens a dialog to upload a specification file.
	Open	Opens a file browser dialog to select and load an existing PDF specification.
	Save	Saves the current state or configuration data.
	Print	Sends the current active report or view to the printer.
Login	(Direct Action)	Displays a modal dialog or panel overlay prompting for user credentials. Authenticates the session.
Help	User Manual	Opens the compiled help file (.chm) or redirects to a web-based documentation URL.
	About	Displays a dialog box showing version number, copyright, and system information.
Data Transfer	Upload	Initiates a transfer protocol (upload of logs or configs).
	Download	Download of logs or configs.

3.2. Left Navigation Panel

This panel contains a vertical stack of selectable command buttons. Only one view can be active at a time.

Button ID	Action on Click
Home	Resets the main content area to display the Company Logo and welcome message.
Node	Loads Node Creation and other functionalities.
Area	Initiates the area functionalities for the respective nodes.
Tests	Loads the Test Execution Module (list of available hardware/software tests).
Settings	Loads the Configuration Panel (network params, thresholds, user preferences).
Network	Loads the Network Status View (IP config, adapter status, ping results).
I/O Activity	Loads the Input/Output Monitor (real-time graph or log of system I/O).
Upload/Download	Loads the File Transfer Manager interface.

3.3. Main Content Area

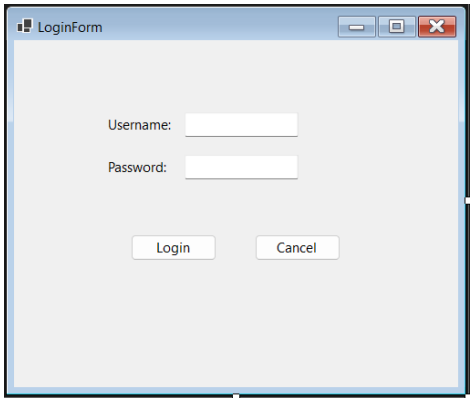
- Default State (Home): Displays a Picture Box containing the company logo (centred). This is the view presented upon application launch.
- Active State: When a button in the Left Navigation Panel is clicked, the content of this area is replaced entirely with the corresponding User Control or Form for that module.

4. Detailed Explanation of Panel Navigation Tabs

The left panel of the main application window contains a vertical stack of command buttons. Each button corresponds to a distinct functional module of the application. Clicking a button dynamically loads the associated view into the central content area, replacing the previously displayed content.

4.1. Login Window Form

The Login Form is a modal dialog that authenticates the user before granting access to protected features. It validates credentials against a locally stored JSON file.



4.1.1 Component Template

Component	Type	Purpose
lblUsername	Label	Labels the username field.
txtUsername	TextBox	Input field for username.
lblPassword	Label	Labels the password field.
txtPassword	TextBox	Input field for password (masked).
btnLogin	Button	Submits credentials for validation.
btnCancel	Button	Cancels login and closes the form.

Form-Level Properties:

- FormBorderStyle = FixedDialog – Prevents user resizing.
- StartPosition = CenterParent – Centers the dialog over the main application window.

4.1.2 Credential Storage

Credentials are stored in a JSON file at the following location:

C:\ProgramData\Company Name\ApplicationName\auth.json

The file contains a single administrator account.

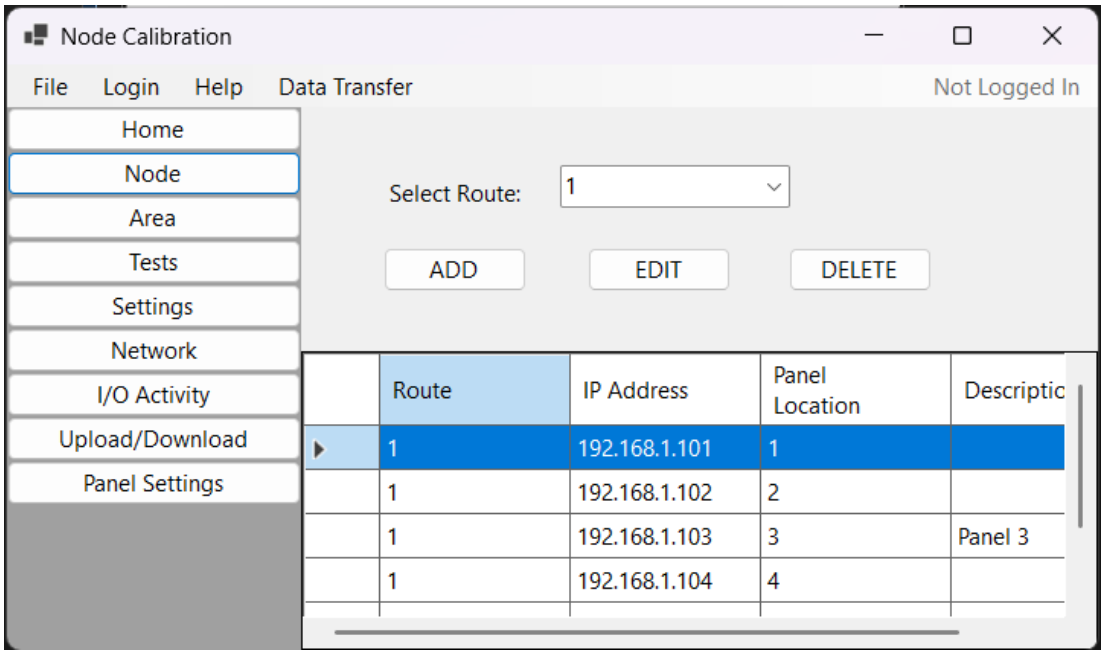
4.1.3 Post-Login Behaviour

Upon successful validation:

- The main menu strip displays a user icon and the text "Administrator".
- The **Login** menu item text changes to **Logout**.

4.2. Nodes User Control Form

The Node Management module allows users to view, add, edit, and delete network node configurations. Data is persisted in a Microsoft SQL Server LocalDB database.

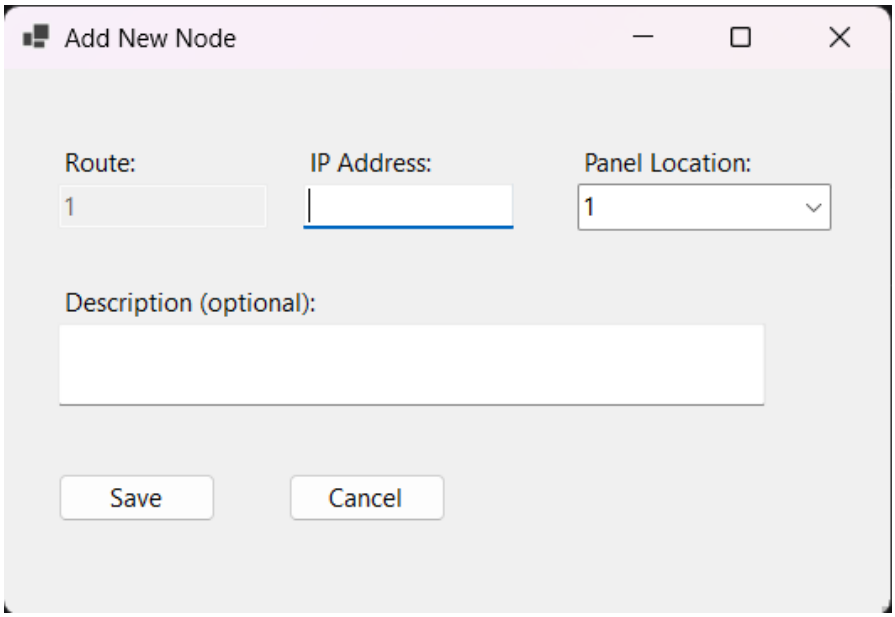


4.2.1 Component Template

Component	Type	Purpose
lblRoute	Label	Labels the route selection dropdown.
cmbRoute	Combo Box	Allows selection of a route (1, 2, 3, or 4).
btnAdd	Button	Opens the Add Node dialog.
btnEdit	Button	Opens the Edit Node dialog for the selected row.
btnDelete	Button	Deletes the selected node after confirmation.
dgvNodes	DataGridView	Displays all existing nodes in a tabular format.

4.2.2 Add / Edit Node Window Form

This dialog is used for both creating a new node and modifying an existing one.



Component	Type	Purpose
lblRoute	Label	Labels the route display field.
txtRoute	Text Box	Displays the selected route (read-only).
lblIP	Label	Labels the IP address input field.
txtIPAddress	Text Box	Accepts an IPv4 address.
lblPanel	Label	Labels the panel location dropdown.
cmbPanelLocation	Combo Box	Accepts a numeric panel location (1–100).
lblDescription	Label	Labels the description field.
txtDescription	Text Box	Accepts optional descriptive text.
btnSave	Button	Saves the node data.
btnCancel	Button	Cancels the operation.

4.2.3 Database Storage

Node records are stored in a SQL Server LocalDB table named Nodes with the following schema:

Column	Data Type	Constraints
Id	int	PRIMARY KEY, IDENTITY(1,1)
Route	int	NOT NULL

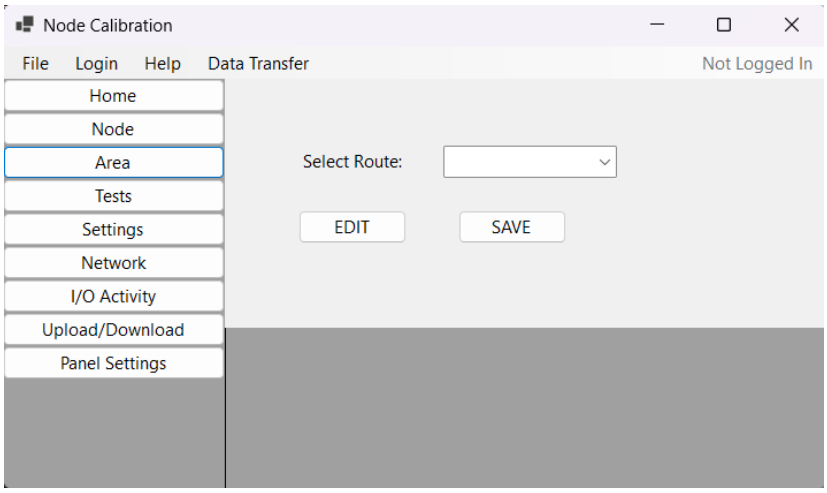
Column	Data Type	Constraints
IPAddress	nvarchar(15)	NOT NULL, UNIQUE
PanelLocation	int	NOT NULL
Description	nvarchar(MAX)	NULL

4.2.5 Functional Workflow

1. **View Nodes:** On loading, the DataGridView is populated with all records from the Nodes table, ordered by PanelLocation.
2. **Add Node:** User selects a route, clicks **ADD**, completes the dialog, and upon successful validation the record is inserted and the grid refreshes.
3. **Edit Node:** User selects a row, clicks **EDIT**, modifies fields in the dialog, and upon validation the record is updated.
4. **Delete Node:** User selects a row, clicks **DELETE**, confirms the action, and the record is removed.

4.3. Area User Control Form

The Area Management module allows the user to assign descriptive labels to each of the 100 panel locations across the four routes. These descriptions help document the physical placement of hardware panels.



4.3.1 Component Template

Component	Type	Purpose
lblRoute	Label	Labels the route selection dropdown.
cmbRoute	Combo Box	Allows selection of a route (1, 2, 3, or 4).
btnEdit	Button	Opens the Edit Description dialog for the selected row.
btnSave	Button	Saves all pending description changes to the database.
dgvAreas	DataGridView	Displays panel location data for the selected route.

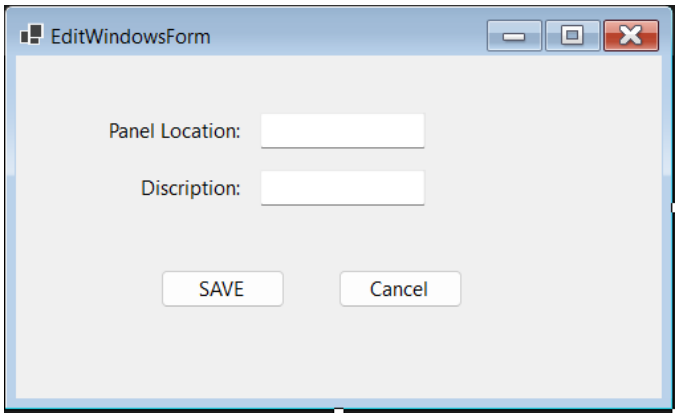
4.3.2 Grid Column Definition

The DataGridView displays the following columns for the selected route:

Column Name	Data Source	Description
Panel Location	PanelLocation	The numeric panel identifier (1–100). This column is read-only.
Description	Description	The user-defined label describing the physical location.

4.3.3 Edit Description Dialog Template

When the user selects a row and clicks **EDIT**, a modal dialog opens with the following components:



Component	**Type**	**Purpose**
lblPanelLocation	Label	Labels the panel location display field.
txtPanelLocation	Text Box	Displays the selected panel location number (read-only).
lblDescription	Label	Labels the description input field.
txtDescription	Text Box	Accepts the descriptive text for the panel location.
btnSave	Button	Saves the description and closes the dialog.
btnCancel	Button	Cancels editing and closes the dialog.

Form-Level Properties:

- FormBorderStyle = FixedDialog
- StartPosition = CenterParent
- Text = "Edit Panel Description"

4.3.4 Functional Workflow

1. **Route Selection:** The user selects a route (1–4) from the dropdown. The grid is populated with 100 rows representing panel locations 1 through 100. If descriptions have been previously saved, they are displayed; otherwise, default values (e.g., A1, A2, ...) may be shown.
2. **Edit Description:**
    - The user selects a row in the grid.
    - Clicking **EDIT** opens the Edit Description dialog. The Panel Location field is pre-filled and non-editable. The current description (if any) is loaded into the text box.
    - The user modifies the description and clicks **Save**. The dialog closes, and the grid is updated to reflect the new description.
3. **Save to Database:**
    - Clicking the **SAVE** button on the main AreaControl commits all pending description changes to the database. This may be a batch save of all modified rows or a single save operation depending on implementation.

4.3.5 Database Storage

Panel descriptions are stored in a SQL Server table named Panel Descriptions with the following schema:

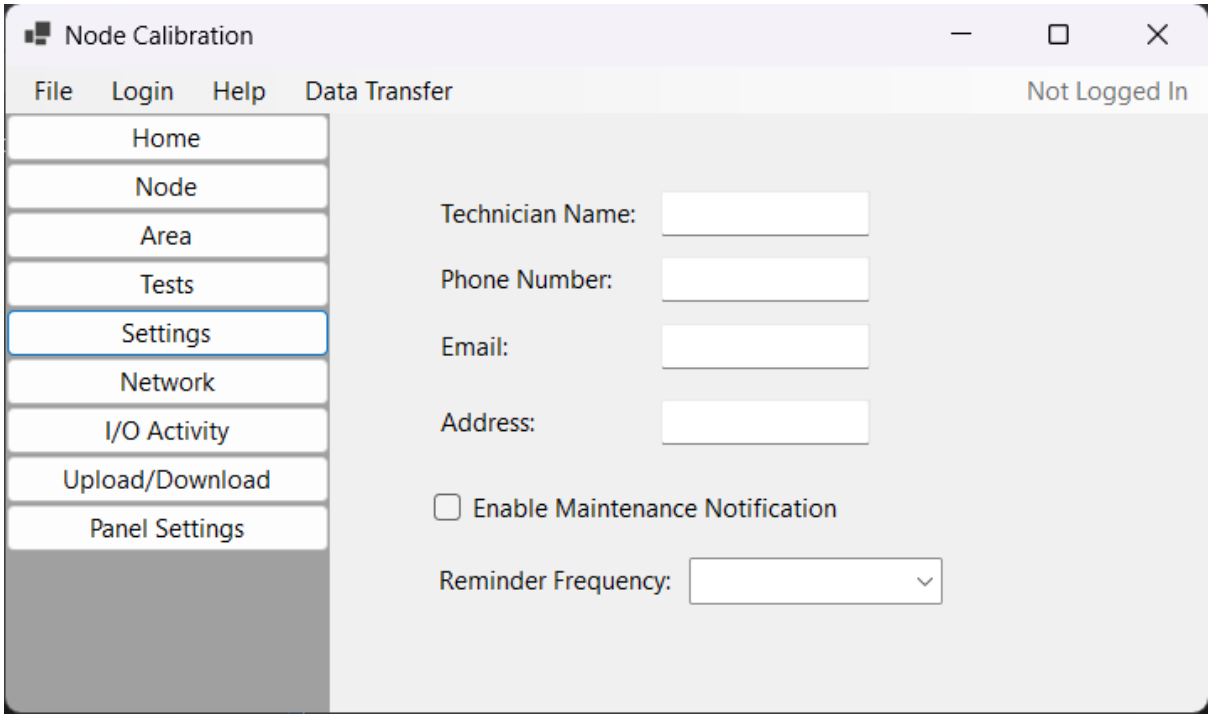
Column	**Data Type**	**Constraints**
Id	int	PRIMARY KEY, IDENTITY (1,1)
Route	int	NOT NULL
PanelLocation	int	NOT NULL
Description	nvarchar(255)	NULL

4.3.6 Relationship to Node Management

The PanelLocation values used in the Node Management module correspond to the same panel locations managed here. The Node module assigns IP addresses to panels, while the Area module provides the human-readable physical description for those same panel slots.

4.4. Settings User Control Form

The Settings module allows the user to configure technician contact information and maintenance notification preferences. These settings are saved locally and persist between application sessions.



4.4.1 Component Template

Component	Type	Purpose
lblTechnicianName	Label	Labels the technician's name input field.

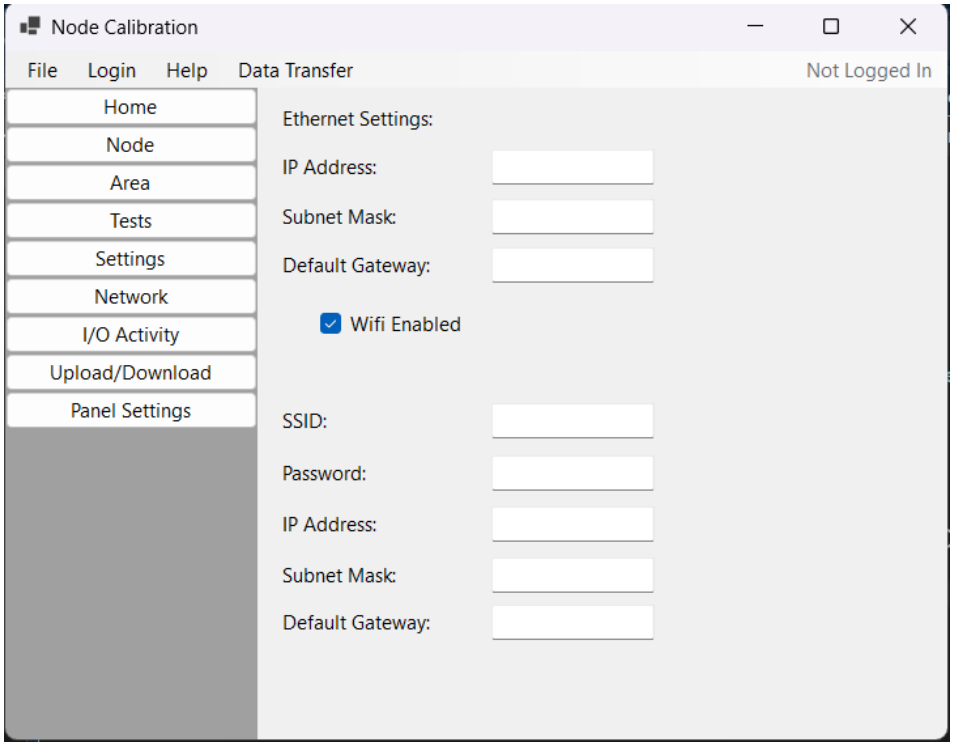
Component	Type	Purpose
txtTechnicianName	TextBox	Stores the name of the responsible technician.
lblPhoneNumber	Label	Labels the phone number input field.
txtPhoneNumber	TextBox	Stores the technician's contact phone number.
lblEmail	Label	Labels the email address input field.
txtEmail	TextBox	Stores the technician's email address.
lblAddress	Label	Labels the physical address input field.
txtAddress	TextBox	Stores the technician's or site address.
CheckNotification	CheckBox	Toggles the maintenance reminder feature.
lblFrequency	Label	Labels the frequency dropdown.
cmbFrequency	ComboBox	Selects the reminder interval in months.

4.4.1 Functional Workflow

1. **Load Settings:** When the SettingsControl is instantiated, it reads the settings.json file (if it exists) and populates the text boxes, checkbox, and dropdown with the saved values. Default values are used if no file exists.
2. **Edit Settings:** The user can modify any of the fields.
3. **Enable/Disable Frequency:** The frequency dropdown is enabled only when the "Enable Maintenance Notification" checkbox is checked. When unchecked, the dropdown is disabled.
4. **Save Settings:** Changes are typically saved automatically when the user navigates away from the Settings module or when an explicit **Save** button is provided (implementation may vary). The updated values are written back to the JSON file.

4.5 Network User Control Form

The Network Configuration module allows the user to view and modify Ethernet and optional Wi-Fi network settings. Ethernet settings are always visible, while Wi-Fi settings are conditionally displayed when enabled via a checkbox.



4.5.1 Component Template

Component	Type	Purpose
lblEthernet	Label	Subheading for wired network settings.
lblEthernetIP	Label	Labels the Ethernet IP field.
txtEthernetIP	Text Box	Displays/accepts Ethernet IPv4 address.
lblEthernetSubnet	Label	Labels the Ethernet subnet mask field.
txtEthernetSubnet	Text Box	Displays/accepts Ethernet subnet mask.
lblEthernetGateway	Label	Labels the Ethernet gateway field.
txtEthernetGateway	Text Box	Displays/accepts Ethernet default gateway.
chkWifiEnabled	Check Box	Toggles visibility of Wi-Fi settings panel.

4.5.3 Functional Workflow

1. **Load Settings:** When the NetworkControl loads, it retrieves saved network configuration from the application settings file. Ethernet fields are populated. If Wi-Fi was previously enabled, the checkbox is checked and the Wi-Fi panel is made visible with its fields populated.
2. **Toggle Wi-Fi Panel:** When the user checks or unchecks, the **Wi-Fi Enabled** checkbox, the pnlWifiSettings panel's Visible property is updated accordingly. When unchecked, any entered Wi-Fi data is retained but hidden.